WHATSAPP · HOME ASSISTANT

WhatsApp-Arbeitswecker

Technische Anleitung zur sicheren, fehlertoleranten Übernahme von Arbeitszeiten in Home Assistant

1	2	3	4
Günna	Meta	Bridge	Home Assistant
WhatsApp-Nachricht	Cloud API + Signatur	prüfen · parsen · speichern	Sensor + Alarm

Version	1.0
Stand	27. Oktober 2026
Projekt	OrbitNestLab/whatsapp-arbeitswecker
Schutzprinzip	Signatur · Whitelist · Deduplizierung · Secrets

Version 1.0 · 27. Oktober 2026 · Projekt: `OrbitNestLab/whatsapp-arbeitswecker`

> Wichtig: Diese Lösung verwendet ausschließlich die offizielle WhatsApp Cloud API. Sie kann nicht heimlich beliebige private Chats oder WhatsApp Web auslesen. Günna sendet die Dienstzeiten an die für die Cloud API eingerichtete WhatsApp-Business-Nummer. Seine Nummer muss als erlaubter Absender konfiguriert und die Verarbeitung mit ihm abgestimmt sein.

1. Ziel und Umfang

Günna sendet eine oder mehrere Arbeitszeiten per WhatsApp, zum Beispiel `Di 05:30, Mi 05:00, Fr 13 Uhr, Sa frei`. Meta übermittelt die Nachricht signiert an eine kleine Bridge. Die Bridge akzeptiert nur Günnas WhatsApp-ID, parst eindeutige Termine, verhindert Doppelverarbeitung und stellt Home Assistant den Plan bereit. Home Assistant löst 60 Minuten vor dem Arbeitsbeginn das vorhandene Skript `script.test_arbeitswecker_fire_hd_start` aus.

Nicht eindeutig auswertbare Nachrichten ändern den gespeicherten Plan nicht. Dadurch kann eine Formulierung wie „morgen etwas früher“ keinen falschen Wecker erzeugen.

2. Architektur und Datenfluss

1	2	3	4
Günna	Meta	Bridge	Home Assistant
WhatsApp-Nachricht	Cloud API + Signatur	prüfen · parsen · speichern	Sensor + Alarm

```
Günnas WhatsApp
        verschlüsselte Nachricht

WhatsApp Cloud API (Meta)
        HTTPS-Webhook + X-Hub-Signature-256

WhatsApp-Arbeitswecker-Bridge
  1. Signatur prüfen
  2. Absender prüfen
  3. Nachrichten-ID deduplizieren
  4. Arbeitszeiten parsen
  5. SQLite aktualisieren
        geschützte REST-Abfrage im lokalen Netz

Home Assistant: sensor.whatsapp_arbeitsplan
        Zeitprüfung jede Minute, 60 Minuten Vorlauf

script.test_arbeitswecker_fire_hd_start

vorhandene Fire-HD-/Weckersteuerung
```

Die öffentliche Angriffsfläche ist auf `POST/GET /webhook` begrenzt. Die Schedule-API `/api/schedule` bleibt idealerweise nur im lokalen Netz erreichbar und verlangt zusätzlich einen Bearer-Schlüssel.

3. Erforderliche Dienste und Integrationen

- Meta-Konto mit Zugriff auf ein Meta-Entwicklerkonto und ein Business-Portfolio
- Meta-App vom Typ „Business“ mit dem Produkt WhatsApp

- WhatsApp Cloud API mit einer eigenen oder Meta-Testtelefonnummer
- Öffentliche HTTPS-Adresse für den Webhook, zum Beispiel über einen abgesicherten Reverse Proxy oder Tunnel
- Docker oder Python 3.11+ auf einem dauerhaft erreichbaren System
- Home Assistant mit aktivierten Packages oder alternativ manueller Übernahme der YAML-Bausteine
- Lokale Erreichbarkeit der Bridge von Home Assistant

Ein direktes Portforwarding vom Internet auf Home Assistant oder Port 8080 der Bridge ist nicht erforderlich und wird nicht empfohlen.

4. Platzhalterübersicht

Alle Werte in eckigen Klammern müssen bei der Einrichtung lokal ersetzt werden. Sie dürfen nicht in Git eingecheckt werden.

- `[META_VERIFY_TOKEN]`: selbst erzeugter langer Zufallswert für die Webhook-Verifizierung
- `[META_APP_SECRET]`: App-Secret der Meta-App
- `[WHATSAPP_SENDER_ID_GUENNA]`: Günnas Nummer im internationalen Format, nur Ziffern, zum Beispiel `49…`
- `[BRIDGE_API_KEY]`: langer zufälliger Schlüssel für Home Assistants Schedule-Abfrage
- `[PUBLIC_BRIDGE_HOST]`: öffentlicher HTTPS-Hostname des Webhooks
- `[BRIDGE_HOST]`: lokaler Hostname oder lokale IP der Bridge
- `[HA_WEBHOOK_URL]`: optionaler vollständiger Home-Assistant-Webhook
- `[SCRIPT_ENTITY]`: falls das Zielskript nicht `script.test_arbeitswecker_fire_hd_start` heißen soll

Zufallswerte können lokal zum Beispiel mit `openssl rand -hex 32` erzeugt werden. Den ausgegebenen Wert niemals in Chat, Tickets oder Git einfügen.

5. WhatsApp Cloud API und Webhook einrichten

1. Unter Meta for Developers eine Business-App erstellen und „WhatsApp“ hinzufügen.
2. Im WhatsApp-Produkt die Business-Telefonnummer einrichten. Für einen Test darf zunächst die Meta-Testnummer verwendet werden.
3. Unter WhatsApp → Configuration die Callback-URL `https://[PUBLIC_BRIDGE_HOST]/webhook` eintragen.
4. Als Verify Token exakt den lokalen Wert `[META_VERIFY_TOKEN]` eintragen.
5. Das Webhook-Feld `messages` abonnieren.
6. Günnas Nummer als erlaubten Absender in `ALLOWED_SENDER_WA_ID` eintragen. Nur Ziffern verwenden.
7. Im Produktivbetrieb die Meta-App live schalten und die von Meta geforderten Prüfungen beziehungsweise Berechtigungen abschließen.

Bei der Verifizierung ruft Meta `GET /webhook` mit `hub.mode`, `hub.verify_token` und `hub.challenge` auf. Die Bridge gibt die Challenge nur zurück, wenn der Token exakt stimmt. Eingehende Nachrichten werden zusätzlich mit dem App-Secret und `X-Hub-Signature-256` geprüft.

Offizielle Links:

- WhatsApp Cloud API: https://developers.facebook.com/docs/whatsapp/cloud-api/

- Einstieg: https://developers.facebook.com/docs/whatsapp/cloud-api/get-started
- WhatsApp-Webhooks: https://developers.facebook.com/docs/whatsapp/cloud-api/webhooks
- Graph-API-Webhooks: https://developers.facebook.com/docs/graph-api/webhooks/getting-started

6. Bridge installieren und konfigurieren

Projekt kopieren, lokale Umgebungsdatei erstellen und ausschließlich dort die echten Werte hinterlegen:

```
cp .env.example .env
chmod 600 .env
```

Beispielinhalt:

```
META_VERIFY_TOKEN=[META_VERIFY_TOKEN]
META_APP_SECRET=[META_APP_SECRET]
ALLOWED_SENDER_WA_ID=[WHATSAPP_SENDER_ID_GUENNA]
BRIDGE_API_KEY=[BRIDGE_API_KEY]
TIMEZONE=Europe/Berlin
DATABASE_PATH=/data/arbeitswecker.db
HA_WEBHOOK_URL=[HA_WEBHOOK_URL]
```

Start mit Docker Compose:

```
docker compose up -d --build
docker compose ps
curl http://127.0.0.1:8080/health
```

Die Compose-Datei bindet Port 8080 nur an `127.0.0.1`. Ein vorgeschalteter HTTPS-Proxy veröffentlicht ausschließlich `/webhook`. Muss Home Assistant von einem anderen Rechner auf `/api/schedule` zugreifen, darf der Port gezielt im vertrauenswürdigen LAN gebunden oder der Pfad über einen internen Proxy angeboten werden. Keinesfalls ungeschützt ins Internet stellen.

7. Nachrichtenverarbeitung und Parser

Unterstützte Beispiele:

- `Di 05:30`
- `Dienstag 530`
- `Fr 13 Uhr`
- `27.10 07:15`
- `Sa frei` oder `Sa Urlaub`
- `Di 05:30, Mi 05:00; Fr 13 Uhr; Sa frei`

Wochentage werden in der Zeitzone `Europe/Berlin` auf das nächste passende Datum aufgelöst. Liegt die Zeit am aktuellen Wochentag bereits in der Vergangenheit, wird die folgende Woche verwendet. Ein Datum ohne Jahr wird auf das nächste plausible Datum aufgelöst.

Sicherheits- und Konsistenzregeln:

- Nur Textnachrichten werden ausgewertet.
- Nur `[WHATSAPP_SENDER_ID_GUENNA]` darf den Plan ändern.

- Jede WhatsApp-Nachrichten-ID wird nur einmal verarbeitet.
- Pro Datum gewinnt die zuletzt erfolgreich verarbeitete eindeutige Angabe.
- Unklare Texte werden protokolliert, ändern aber keine Termine.
- Der Nachrichtentext wird nicht dauerhaft gespeichert; die Datenbank enthält Status, Termin und eine begrenzte Diagnose.

8. Home-Assistant-Anbindung

Die Datei `home_assistant/package_whatsapp_arbeitswecker.yaml` nach `/config/packages/` kopieren. In `/config/secrets.yaml` ergänzen:

```
whatsapp_arbeitswecker_url: "http://[BRIDGE_HOST]:8080/api/schedule"
whatsapp_arbeitswecker_api_key: "[BRIDGE_API_KEY]"
```

Packages müssen in `configuration.yaml` aktiviert sein:

```
homeassistant:
  packages: !include_dir_named packages
```

Danach zuerst die Konfiguration prüfen. Erst bei erfolgreicher Prüfung neu laden beziehungsweise kontrolliert neu starten.

Das Package erzeugt:

- `sensor.whatsapp_arbeitsplan` mit `entries`, `count`, `version` und `updated_at`
- `input_boolean.whatsapp_arbeitswecker_aktiv`
- `input_text.whatsapp_arbeitswecker_letzte_ausloesung` zur Deduplizierung
- Automation `WhatsApp Arbeitswecker  60 Minuten vorher`

Die vorhandene Entität `sensor.arbeitsplan_marcel` und bestehende Weckerautomationen werden bewusst nicht überschrieben. Nach einem kontrollierten Paralleltest kann das Dashboard optional auf `sensor.whatsapp_arbeitsplan` umgestellt werden. Das Zielskript ist bereits passend zur vorhandenen Anlage auf `script.test_arbeitswecker_fire_hd_start` gesetzt; bei einer anderen Installation muss es durch `[SCRIPT_ENTITY]` ersetzt werden.

Offizielle Home-Assistant-Links:

- RESTful Sensor: https://www.home-assistant.io/integrations/sensor.rest/
- Packages: https://www.home-assistant.io/docs/configuration/packages/
- Automation Trigger: https://www.home-assistant.io/docs/automation/trigger/
- REST API: https://developers.home-assistant.io/docs/api/rest/

9. Öffentlicher HTTPS-Zugang

Der Meta-Webhook muss aus dem Internet per gültigem HTTPS-Zertifikat erreichbar sein. Empfohlene Varianten:

1. Cloudflare Tunnel oder ein vergleichbarer ausgehender Tunnel nur zur Bridge.
2. Bestehender Reverse Proxy mit TLS, Rate-Limit und Weiterleitung ausschließlich von `/webhook`.

3. Ein kleiner externer Server, der die Bridge betreibt und die Schedule-Daten nur über VPN/Tailscale an Home Assistant liefert.

Keine Routerfreigabe direkt zu Home Assistant einrichten. Der Proxy soll Request-Größen begrenzen, TLS erzwingen und keine Debug-Ausgaben veröffentlichen. Die Meta-Signaturprüfung bleibt auch hinter dem Proxy zwingend aktiv.

10. Sicherheitsmaßnahmen

- `.env`, Datenbank und Home-Assistant-Secrets sind durch `.gitignore` ausgeschlossen.
- Tokens mindestens 32 zufällige Bytes lang wählen und getrennt verwalten.
- Meta-App-Secret niemals als Verify Token wiederverwenden.
- Absender-ID strikt auf Günna begrenzen.
- Webhook-Signatur vor JSON-Verarbeitung prüfen.
- Öffentliche Route auf `/webhook` beschränken; `/api/schedule` nur intern.
- Container läuft ohne Root-Rechte und mit minimalem Image.
- Updates kontrolliert einspielen und vorher Datenbank sichern.
- Logs ohne Nachrichtentexte und Geheimnisse betreiben.
- Günnas Einwilligung und die geltenden Datenschutzregeln beachten.
- Schlüssel nach Verdacht oder Personalwechsel rotieren.

11. Testplan

Automatisierte Tests

```
python -m pip install -e '.[test]'
pytest
```

Geprüft werden Parserformate, Datumswechsel, unklare Texte, Webhook-Verifizierung, Signaturablehnung, unbekannte Absender und Deduplizierung.

Manueller Abnahmetest

1. `/health` liefert `{"status":"ok"}`.
2. Meta akzeptiert Callback-URL und Verify Token.
3. Günna sendet `Di 05:30, Mi frei`; der Sensor enthält zwei passende Einträge.
4. Dieselbe Webhook-Nachricht erneut zustellen; `count` bleibt unverändert.
5. Nachricht von einer anderen Nummer senden; der Plan bleibt unverändert.
6. Nachricht `morgen früher` senden; der Plan bleibt unverändert und ein Parserhinweis erscheint im Log.
7. Falsche Signatur senden; HTTP 401 wird geliefert.
8. Testtermin auf etwa 61 Minuten in der Zukunft setzen und prüfen, ob genau einmal das vorhandene Weckerskript startet.
9. Bridge kurz neu starten; der Plan bleibt durch SQLite erhalten.

10. Netzwerk zur Bridge unterbrechen; Home Assistant zeigt den Sensor als nicht verfügbar, löst aber keinen falschen Wecker aus.

12. Fehlerszenarien und Diagnose

Meta meldet „Callback URL konnte nicht validiert werden“ HTTPS-Zertifikat, öffentliche DNS-Auflösung, Proxy-Pfad und identischen Verify Token prüfen. `GET /webhook` muss die Challenge als Klartext liefern.

Webhook liefert HTTP 401 `META_APP_SECRET` gehört zur tatsächlich sendenden Meta-App. Proxy darf Body und Signaturheader nicht verändern.

Nachricht wird ignoriert Die eingehende `from`-ID mit `[WHATSAPP_SENDER_ID_GUENNA]` vergleichen. Nur Ziffern verwenden. Keine echte ID ins Log oder in Git kopieren.

Parser erkennt keine Zeit Ein unterstütztes eindeutiges Format verwenden. Unklare Angaben sind absichtlich fehlersicher und überschreiben nichts.

Home Assistant zeigt „unavailable“ Bridge-Erreichbarkeit, URL, Bearer-Schlüssel und Containerstatus prüfen. Die Schedule-API muss vom Home-Assistant-Rechner erreichbar sein.

Wecker startet doppelt `input_text.whatsapp_arbeitswecker_letzte_ausloesung` und Automationstraces prüfen. Bei parallelem Test sicherstellen, dass nicht zusätzlich eine alte Automation denselben Termin auslöst.

Zeit ist um eine Stunde verschoben `TIMEZONE=Europe/Berlin`, Systemzeit und Zeitzone in Home Assistant kontrollieren. Die Python-Zonenbibliothek berücksichtigt Sommer- und Winterzeit.

13. Betrieb, Backup und Aktualisierung

Regelmäßig die SQLite-Datei `/data/arbeitswecker.db` sichern. Vor einem Update:

```
docker compose ps
cp data/arbeitswecker.db [SICHERER_BACKUP_PFAD]/arbeitswecker-YYYYMMDD.db
docker compose build --pull
docker compose up -d
docker compose ps
```

Nach jedem Update `/health`, eine signierte Testnachricht und den Home-Assistant-Sensor prüfen. Alte Backups nicht automatisch löschen, bevor ihre Wiederherstellbarkeit geklärt ist.

14. Grenzen der Lösung

- Die Cloud API empfängt nur Nachrichten an die eingerichtete WhatsApp-Business-Nummer; sie liest keinen bestehenden privaten Chatverlauf.
- Medien, Sprachnachrichten und frei formulierte Aussagen werden nicht interpretiert.
- Die Bridge plant den Arbeitsbeginn, nicht die tatsächliche Anwesenheit oder GPS-Position.
- Ausgehende automatische Antworten sind nicht aktiviert. Dafür gelten zusätzliche WhatsApp-Regeln, Vorlagen und das 24-Stunden-Fenster.
- Die produktive Meta-Freischaltung und Telefonnummernverifizierung müssen durch den Kontoinhaber erfolgen.

15. Übergabe-Checkliste

- ☐ Meta-App und Business-Telefonnummer eingerichtet
- ☐ Öffentliche HTTPS-Callback-URL erreichbar
- ☐ `messages`-Webhook abonniert
- ☐ Alle Platzhalter nur lokal ersetzt
- ☐ `.env` mit Dateirecht 600 und nicht in Git
- ☐ Absender-ID von Günna bestätigt
- ☐ Bridge-Healthcheck erfolgreich
- ☐ Home-Assistant-Konfiguration geprüft
- ☐ Paralleltest ohne Eingriff in `sensor.arbeitsplan_marcel` erfolgreich
- ☐ Einmalige Auslösung 60 Minuten vorher bestätigt
- ☐ Backup- und Schlüsselrotation dokumentiert